

COMP3105- Fall 2025

Assignment 3

Q1-

(a) minMulDev Implementation

The multinomial deviance loss used for multi-class logistic regression is:

$$W^* = \arg \min_{W \in \mathbb{R}^{d \times k}} \frac{1}{n} \sum_{i=1}^n \left[\log \left(\mathbf{1}_k^\top e^{W^\top x_i} \right) - y_i^\top W^\top x_i \right].$$

We implemented minMulDev using `scipy.optimize.minimize`, computing gradients explicitly and using `scipy.special.logsumexp` to ensure numerical stability.

(b) classify Implementation

Predictions are generated using:

$$\widehat{Y}_{\text{test}} = \text{indmax}(X_{\text{test}} W),$$

where `indmax` applies `argmax` to each row and outputs a one-hot encoded label.

(c) calculateAcc Implementation

Accuracy is computed by comparing predicted and true labels:

$$\text{acc} = \frac{1}{m} \sum_{i=1}^m \mathbb{I}(\arg \max(\widehat{y}_i) = \arg \max(y_i)).$$

(d) Synthetic Experiments

Training Accuracies

n	Model 1	Model 2
16	0.969	1.000
32	0.931	0.988
64	0.887	0.956
128	0.866	0.940

Test Accuracies

n	Model 1	Model 2
16	0.714	0.865
32	0.779	0.905
64	0.823	0.904
128	0.842	0.919

(e)

Model 2 consistently outperforms Model 1, indicating better separation between its four clusters.

Increasing n improves test accuracy for both models and reduces the train–test gap.

Model 1 overfits more strongly, especially for small datasets (e.g, $0.969 \rightarrow 0.714$ at $n = 16$).

Model 2 generalizes much better.

Q2-

(a) PCA Implementation

Steps:

1. Center data:

$$\tilde{X} = X - \mu, \text{ where}$$

$$\mu = \frac{1}{n} \sum_{i=1}^n x_i$$

2. Compute eigenvectors of $\tilde{X}^T \tilde{X}$
3. Return matrix U of top k principal directions

(b) projPCA Implementation

Projection is:

$$X_{\text{proj}} = (X_{\text{test}} - \mu^{\top})U^{\top}.$$

Broadcasting ensures efficient computation.

(c) kernelPCA Implementation

1. Compute kernel matrix K
2. Center it:

$$\widetilde{K} = K - \frac{1}{n}1K - \frac{1}{n}K1 + \frac{1}{n^2}1K1$$

3. Compute eigenvectors α , normalized by

$$\alpha = \frac{v}{\sqrt{\lambda n}}$$

(d) projkernelPCA Implementation

Test projection uses:

$$Y_{\text{test}} = \widetilde{K}_{\text{te,tr}}A^{\top}.$$

(e) PCA Classification Experiments

Training Accuracy

Dim K	Model 1	Model 2
1	0.85578	0.63820
2	0.85578	0.93906

Test Accuracy

Dim K	Model 1	Model 2
1	0.84438	0.62899
2	0.83259	0.91576

(f) Analysis

- Model 1 loses almost no accuracy when reduced to 1D.
- Model 2 collapses in 1D, meaning both principal directions contain essential class-separating information.
- PCA is useful for Model 1 but not ideal for Model 2 unless keeping at least $k = 2$ components.

Q3-

(a) kmeans Implementation

Iterative updates:

1. Assignment step:
Assign each point to its nearest cluster
2. Update step:

$$U^* = (Y^T Y)^{-1} Y^T X = Y^+ X$$

3. Objective:

$$J(Y, U) = \frac{1}{2n} \|X - YU\|_F^2$$

(b) repeatKmeans Implementation

Run k-means multiple times (100 restarts) and select the clustering with the minimum objective value to avoid poor local minima.

(c) chooseK Implementation

K	2	3	4	5	6	7	8	9	
Obj-val	2.640	1.617	0.869	0.710	0.594	0.513	0.452	0.400	

The method indicates an optimal choice around $K=4$.
Improvement beyond 4 clusters is marginal.

(e) kernelKmeans Implementation

Kernel k-means uses the distance:

$$D = \text{diag}(K) \mathbf{1}_k^\top + \mathbf{1}_n \text{diag}(Y^+ K (Y^+)^{\top})^\top - 2K(Y^+)^{\top}.$$

This allows clustering in non-linear feature spaces.

Analysis

- K=4 aligns with the true generative model (four clusters).
- Kernel k-means successfully handles non-linearly separable shapes.
- Both methods provide robust clustering capability, with kernel k-means offering greater flexibility.